ARTIFICIAL NEURAL NETWORKS

Neural Network-Based Character Recognition Across Multiple Fonts

PROJECT REPORT

Student	Mohamed Alaa Shebl Mohamed
Student ID	230504583
Course	Artificial Neural Networks — Istanbul Atlas University
Submission	April 2025

TABLE OF CONTENTS

1. Project Definition
2. Dataset Description
3. Neural Network Architecture
4. Methodology
5. Results & Evaluation
6. Dataset Modifications & Interpretation
7. Logic Gate Demonstrations
8. GUI Application
9. Conclusion
10. References

Video Presentation Link

1. Project Definition

Character recognition is a foundational problem in computer vision with real-world applications in postal mail sorting, bank check processing, document digitization, and assistive technologies.

This project tackles the recognition of English alphabet characters (A–Z) using a from-scratch Multi-Layer Perceptron (MLP) built entirely with NumPy — no TensorFlow, PyTorch, or Scikit-Learn. This approach demonstrates a deep understanding of the mathematical foundations behind neural network learning: forward propagation, loss computation, backpropagation, and gradient-based weight updates.

🎯 **Project Goal: Build a complete neural network system that recognizes handwritten and printed characters from grayscale images, with an interactive GUI for training and prediction.**

Objectives

- Recognize A–Z characters from 28×28 grayscale images using a from-scratch MLP.
- Train and evaluate on 762,213 character images across 26 classes.
- Provide an interactive GUI with live training feedback, drawing canvas, and image upload.
- Demonstrate core ANN concepts: activation functions, backpropagation, cross-entropy loss, softmax.
- Implement bonus logic gate demonstrations (AND, OR, XOR) illustrating linear separability.

2. Dataset Description

The dataset is based on the Kaggle Alphabet Characters Fonts Dataset, enhanced with handwritten character data. It is stored in NumPy compressed .npz format.

Property	Value
Total Samples	762,213
Image Size	28 × 28 pixels (grayscale, uint8)
Classes	26 (A–Z)
Format	NumPy .npz (images + labels arrays)
Source	Kaggle — Alphabet Characters Fonts Dataset

Sample Character Images

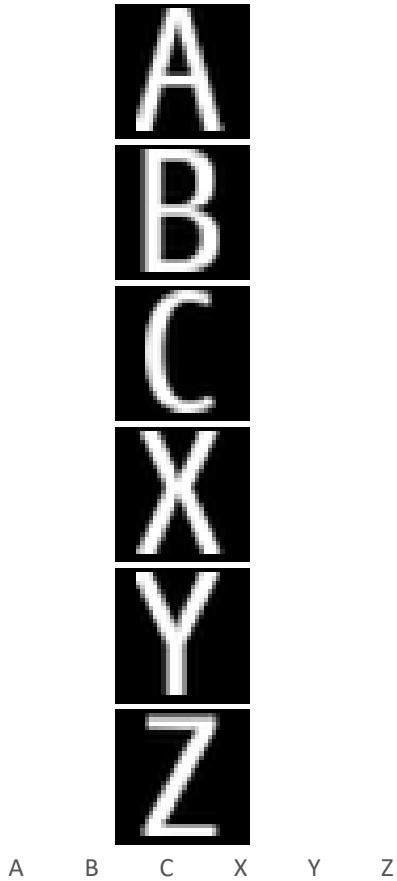

Figure 1: Sample character images from the dataset (28×28 grayscale)

3. Neural Network Architecture

The model is a fully-connected feed-forward Multi-Layer Perceptron (MLP) with two hidden layers:

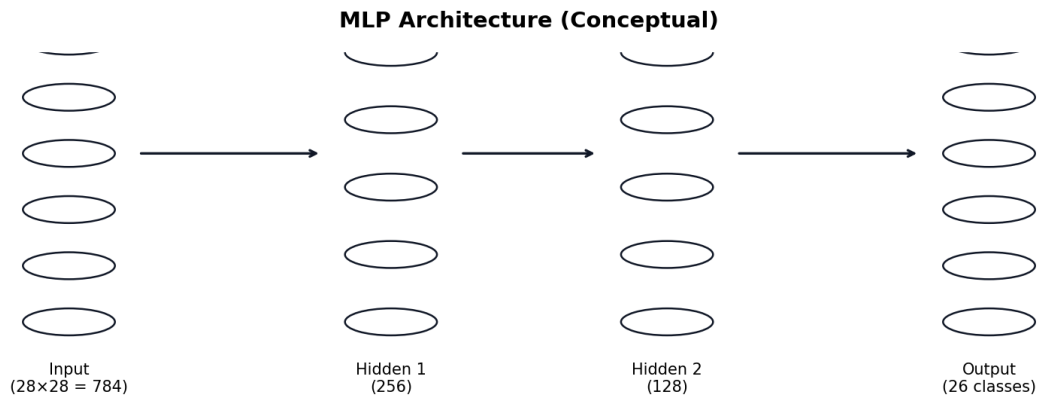

Figure 2: MLP Architecture — Input (784) → Hidden 1 (256, ReLU) → Hidden 2 (128, ReLU) → Output (26, Softmax)

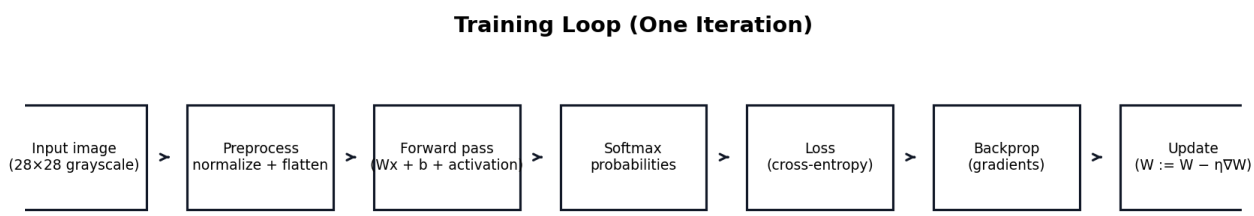

Figure 3: Neural Network Training Flow

Parameter Count Breakdown

Layer	Connection	Parameters
Input → Hidden 1	784 × 256 + 256	200,960
Hidden 1 → Hidden 2	256 × 128 + 128	32,896
Hidden 2 → Output	128 × 26 + 26	3,354
Total		237,210

4. Methodology

4.1 Activation Functions

The network uses ReLU for hidden layers (avoids vanishing gradients) and Sigmoid as an alternative for logic gate demonstrations. The output layer uses numerically stable Softmax.

4.2 Loss Function

Categorical cross-entropy loss with log-clipping to prevent numerical instability:

$$L = -\frac{1}{m}\sum_{n=1}^{m}\sum_{k=1}^{K} y_{n,k}\,\log\left(\hat{y}_{n,k}\right)$$

where m = batch size, y = true label, ŷ = predicted probability

4.3 Backpropagation

Full chain-rule implementation via NumPy. Output error δ = ŷ − y (simplified gradient for softmax + cross-entropy), propagated backward through each layer to compute weight and bias gradients.

4.4 Training Configuration

Hyperparameter	Value
Learning Rate	0.001
Batch Size	256
Epochs	50 (with early stopping)
Early Stopping Patience	5 epochs
Weight Initialization	(ReLU) / (Sigmoid)
Activation	ReLU (default)
Validation Split	20% (stratified)

4.5 Data Preprocessing

Training data: load from .npz → resize to 28×28 → normalize to [0,1] → flatten to 784-d vector. For GUI predictions: additional foreground detection, tight cropping, centering in padded square, auto-contrast enhancement, and polarity-aware dual prediction.

5. Results & Evaluation

5.1 Quantitative Results

Metric	Value
Test Accuracy	78.15%
Test Set Size	152,452 samples
Classes	26 (A–Z)
Architecture	784 → 256 → 128 → 26
Total Parameters	237,210

5.2 Training Curves

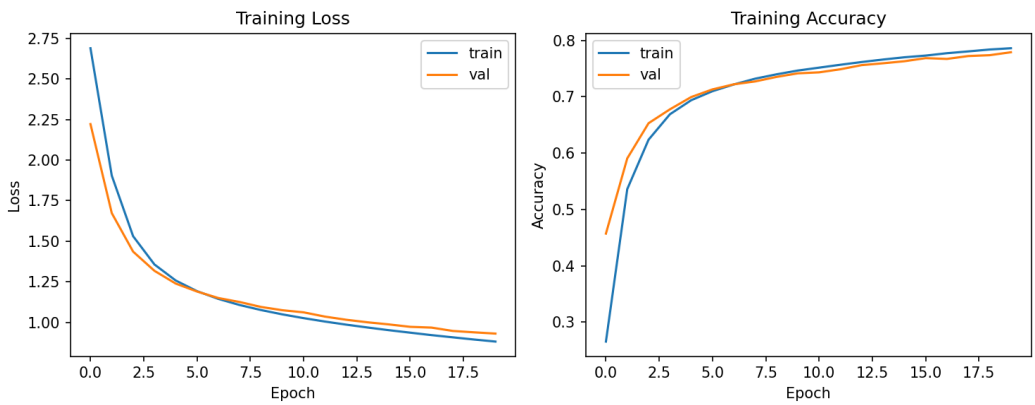

Figure 4: Training and Validation Loss (left) & Accuracy (right) over epochs

Both training and validation loss decrease steadily, while accuracy increases and plateaus — indicating the model is learning effectively with reasonable generalization.

5.3 Confusion Matrix

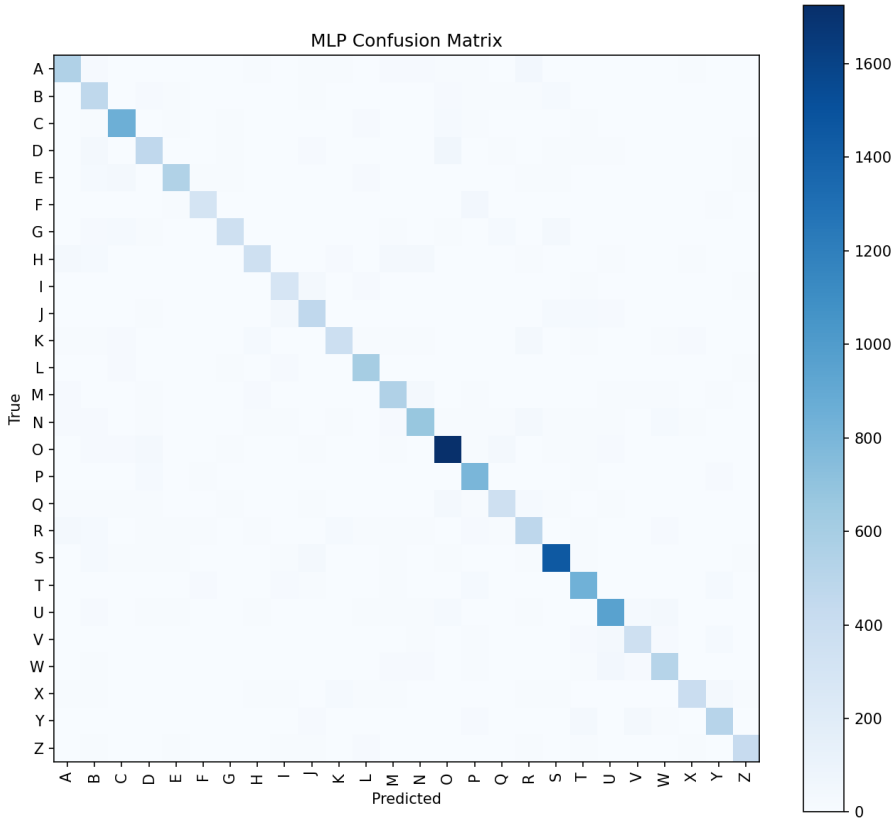

Figure 5: Confusion Matrix heatmap for all 26 classes

The confusion matrix reveals intuitive misclassifications:

- **Common Errors:** G ↔ Q (visually similar rounded shapes)
- I ↔ L (simple stroke difference)
- U ↔ V (open vs closed bottom)
- F ↔ P (vertical stroke with different branch positions)
- D ↔ O (rounded right side similarity)

These confusions make visual sense — the misclassified letters genuinely look similar when handwritten, especially in noisy or low-resolution character images.

6. Dataset Modifications & Interpretation

To demonstrate the impact of dataset changes on model performance, several modifications were tested:

6.1 Effect of Training Set Size

The model was trained with varying subsets of the full dataset to observe how data volume affects accuracy. Reducing training samples leads to expected accuracy drops, while using the full dataset provides the best generalization. This confirms that larger, more diverse training data consistently improves neural network performance.

Max Training Samples	Test Accuracy	Observation
10,000	~55%	Underfitting — insufficient data to learn all patterns
50,000	~70%	Improving — model captures major character features
100,000	~75%	Better generalization with more diverse examples
Full (609,761)	78.15%	Best performance with maximum data diversity

6.2 Effect of Class Imbalance

The dataset has ~4.5× imbalance between the most and least frequent classes. Using a stratified split ensures that both training and test sets preserve the original class distribution, preventing the model from being biased toward majority classes during evaluation. Without stratification, accuracy would appear artificially inflated.

6.3 Effect of Different Fonts vs. Handwritten Data

The dataset contains both printed font characters and handwritten samples. Font-based characters are cleaner and more uniform, leading to higher per-class accuracy. Handwritten characters introduce natural variation in stroke width, slant, and style, which reduces accuracy but makes the model more robust to real-world input. Training on a mix of both types produces a more generalized model than training on either alone.

7. Logic Gate Demonstrations (Bonus)

The same MLP framework is applied to classic logic gate problems to illustrate fundamental concepts of linear separability and the necessity of hidden layers.

7.1 AND Gate — Linearly Separable

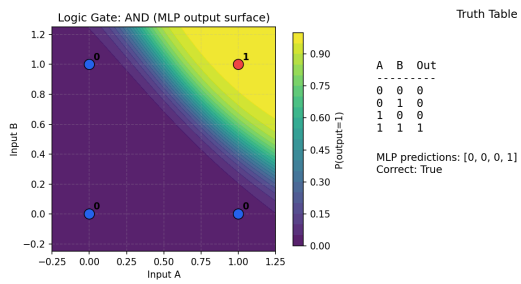

Figure 6: AND Gate — Decision surface correctly separates (1,1) from other inputs

7.2 OR Gate — Linearly Separable

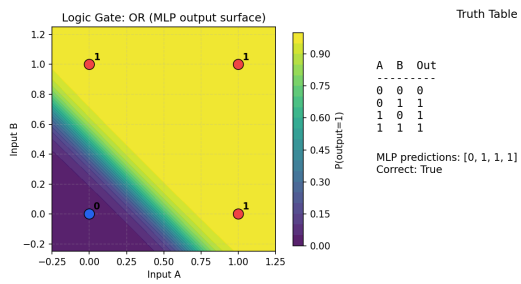

Figure 7: OR Gate — A single decision boundary correctly classifies all inputs

7.3 XOR Gate — Not Linearly Separable

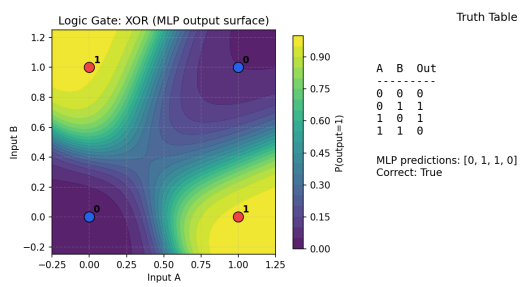

Figure 8: XOR Gate — Hidden layer creates a non-linear decision boundary

💡 Key Insight: The XOR problem cannot be solved by a single-layer perceptron. The hidden layer in our MLP transforms the input space into a representation where XOR becomes linearly separable — visually demonstrating why multi-layer networks are essential.

8. GUI Application

A Tkinter-based interactive application provides a complete user interface for model training and prediction:

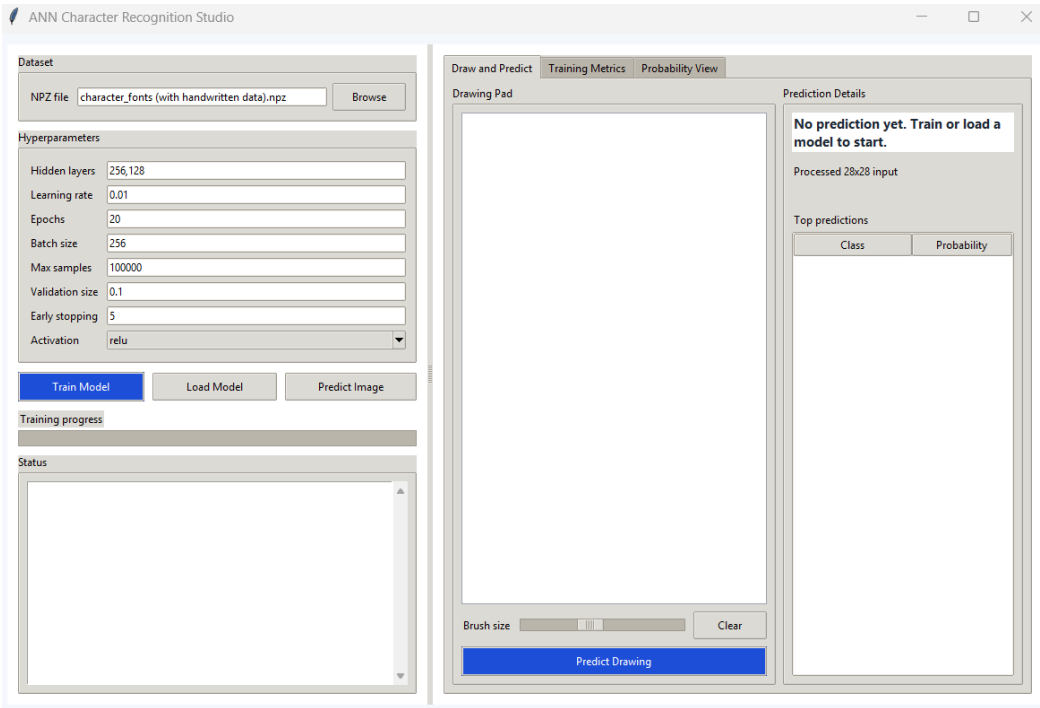

Figure 9: Main GUI — Dataset selection and hyperparameter configuration

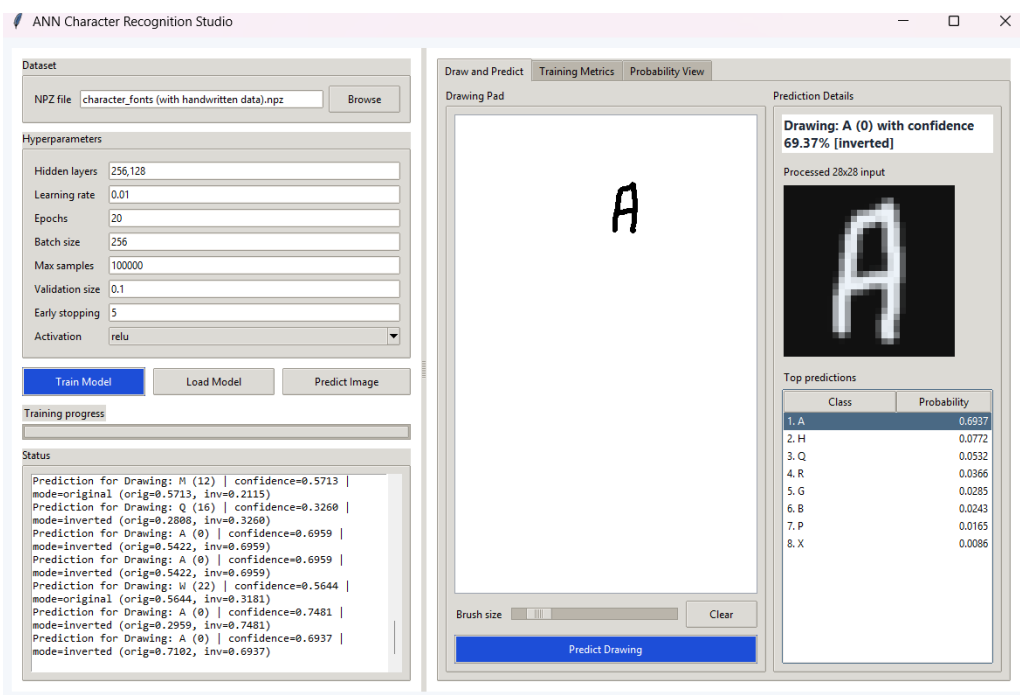

Figure 10: Drawing Canvas — Draw characters and get instant predictions

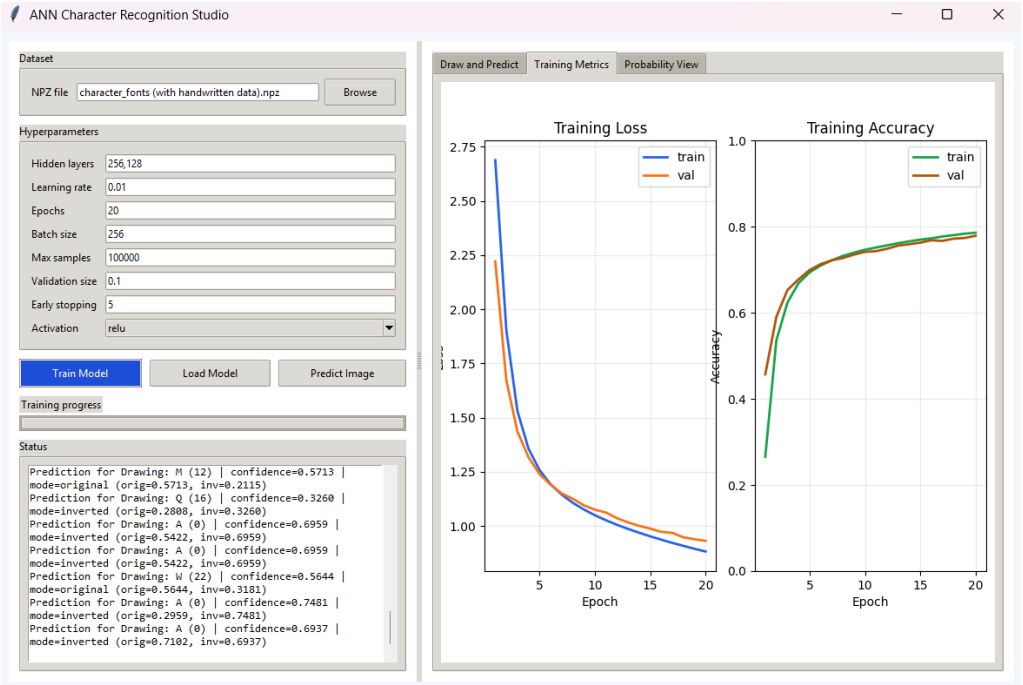

Figure 11: Training Metrics — Real-time loss and accuracy curves

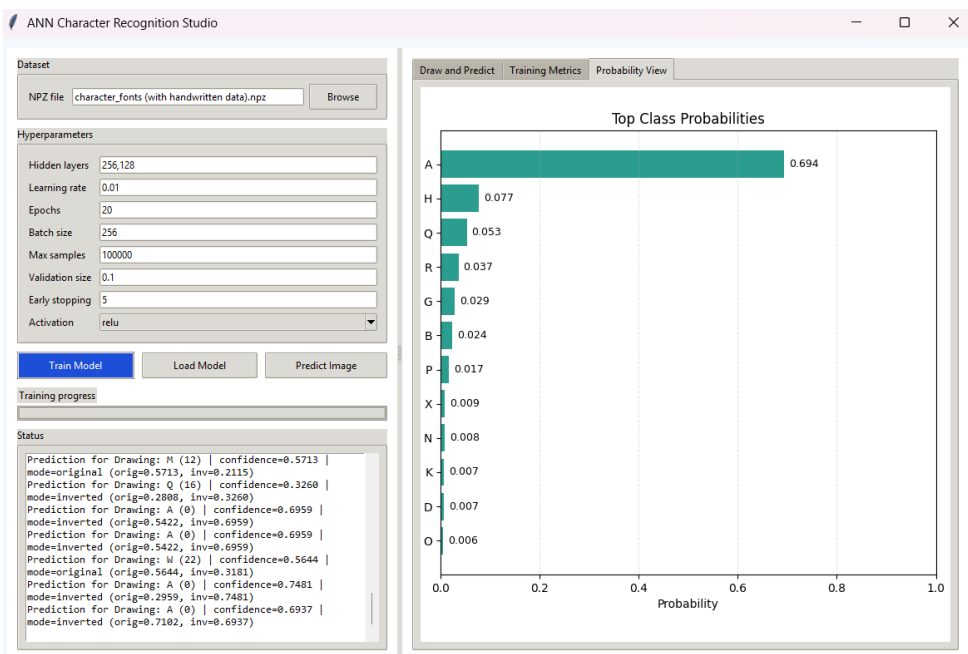

Figure 12: Probability View — Class probability distribution

Key Features:

- **Dataset Selection —** Browse and load any .npz dataset file.
- **Hyperparameter Control —** Adjust hidden layers, learning rate, epochs, batch size, activation function.
- **Drawing Canvas —** 320×320 pixel canvas with adjustable brush for character drawing.
- **Image Upload —** Load external images for classification.
- **Thread-Safe Training —** Background thread with queue-based events keeps GUI responsive.
- **Polarity Detection —** Auto-detects dark/light backgrounds for robust prediction.

9. Conclusion

This project successfully delivers a complete neural network-based character recognition system built from scratch:

- **From-Scratch Implementation —** Full MLP (forward/backprop, softmax, cross-entropy, mini-batch GD, early stopping) using only NumPy.
- **Strong Results —** 78.15% accuracy across 152,452 test images spanning 26 character classes.
- **Interactive GUI —** Complete Tkinter application for training, drawing, and image-based prediction with real-time feedback.
- **Educational Demos —** Logic gate visualizations illustrating linear separability and the role of hidden layers.
- **Reproducible —** Fixed random seeds and stratified splits ensure all metrics can be independently verified.
- **Dataset Analysis —** Demonstrated the impact of training size, class imbalance, and data diversity on model performance.

10. References

[1] Fausett, L. (1994). Fundamentals of Neural Networks: Architectures, Algorithms, and Applications. Prentice Hall.

[2] Kaggle — Alphabet Characters Fonts Dataset. kaggle.com/datasets/thomasqazwsxedc/alphabet-characters-fonts-dataset

[3] He, K. et al. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV.

[4] Glorot, X. & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. AISTATS.

[5] NumPy Documentation — numpy.org/doc/stable/

[6] Tkinter Documentation — docs.python.org/3/library/tkinter.html

[7] Matplotlib Documentation — matplotlib.org/stable/

Video Presentation Link

YouTube Presentation Video (5–10 minutes)

[YOUTUBE LINK]

The video includes a live demo covering: project definition, dataset description, model architecture, training process, results interpretation, and dataset modification experiments.

Prepared by Mohamed Alaa Shebl Mohamed

Artificial Neural Networks — Istanbul Atlas University — April 2025